FRED TALKS

25th August 2026

CONTENTS

Ownership

THORSTEN BALL · 08 JUL 2026 · SOURCE

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It's only lightly edited.

I shared it before, but not here, because I didn't think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don't I? So here we are.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.

Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase "ownership" or allude to it, but haven't explained what "ownership" means to me in a while.

So, ownership.

If I ask you "can you own this?" or "can you take care of this?" or "are you on it?" — what I'm doing is I'm asking you to *own* it, to own the solution of a problem from end to end. From "we have a problem" to "we don't have to think about it again."

That means, when you say that you're owning something, the expectation is that you…

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we're actually trying to solve here. Maybe you think "the problem is that we need to migrate from using X to using Y", but that's not a problem, that's a solution. The problem is likely something like "performance is bad", "it's not stable", "it fails for customer x". Maybe there's other possible solutions to that? Think about those. What are the tradeoffs? What's the best solution to go with considering these tradeoffs?

- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?

- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?

- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven't confirmed yet?

- Think about how you'd test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?

- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!

- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say "here's what I built, under these constraints, with these tradeoffs?"

- Test it manually. Yes, there's automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?

- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it's actually deployed?

- If you think your colleagues needs to know about this change, because it's new feature they should all test, or it's a new convention in codebase, or maybe it's a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.

- Do customers need to know? Who reported the bug? Who's blocked? Let them know.

- Does the world need to know? Announce that it's out.

- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that's a lot. And there's actually more, because I'm sure I forgot some stuff.

But that's how you build a product in a small team. We don't have PMs, we don't have a Q&A department. We're small, but we're *great*, we can do all of that.

And it's always okay to ask for help, it's okay to ask questions, it's okay to redo things and triple-check. What's not okay is to implicitly assume that someone else will do the things here that you haven't thought about.

"How does this apply juniors? You can't expect them to really do all of that?"

I've been asked these questions, or variations of them, after I shared the thoughts above and here's my answer.

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, … — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.

You know what *you* should own? A subscription to this newsletter:

“Code was never the hard part” is an insult to all programmers

SENKO RAŠIĆ · 09 AUG 2026 · SOURCE

The software development profession is in the midst of upheaval. Nobody knows how the AI revolution will play out in the end, but it is clear many aspects of work and life will be transformed—including programming.

One of the comments I hear often lately boils down to “*LLMs may be good at coding, but software was never the hard part*” and “*coding is easy, it's figuring out what to code that's hard*”.

I believe that's a gross insult to all programmers everywhere.

If coding is easy...

If coding is easy, how come programmers were in high demand, and have demanded large salaries for years (even before ZIRP)? Why was there so much stress, overwork and burnout even before AI started churning out 5000-line PRs? Why did companies seek 10x ninja rockstar coders and subject them to leetcode interviews—surely, a junior fresh out of college could churn out something if it's so easy?

If coding is easy, why do we have doorstoppers like Clean Code and The Pragmatic Programmer? Is The Art of Computer Programming a light summer read? Is SICP a coffee-table book? Why do we have bootcamps or even whole college degrees dedicated to it?

If coding is easy, was Carmack just at the right place at the right time? Why do we consider Fabrice Bellard a genius?

If coding is easy, why are people angry at AI (or anyone else) copying their code? Why do they act like they've poured their sweat, soul, and copious amounts of time into something so trivial?

If coding is easy, why do many now feel like their identity and professional purpose are being stripped away from them?

If coding is easy, why is software so damn buggy?

If figuring out what to build is the hard part...

If deciding what to build is the hard part, why do so many product managers seem clueless? Why aren't there rigorous 10-step interviews for them? Why aren't they getting paid more than the developers?

If deciding what to build is the hard part, why aren't market researchers, usability experts and—hell, customer success—considered rockstars in a software company? If “understanding the customer” is harder, why are business analysts looked down on as pencil pushers?

If implementation is easy and finding demand is harder, why are programmers upset when the salespeople promise a new feature to a customer to close the sale? They've found a genuine demand, something people will pay for!

If coding is easy, why doesn't everyone just build ten variations of a thing and see which pans out?

There's no median programmer

Another cliché comment is “*most work in software development is talking to stakeholders, understanding the customer's needs, and having clarity on the priorities*”.

I have met many programmers throughout my career, and very few of them want to talk to stakeholders, much less customers (exceptions are freelancers and founders, especially of software development shops). And, “having clarity on the priorities” boils down to “just tell me what to do and don't switch it up every two days”.

Some software developers do say “I don't write code, I solve customer's problems”. But then they turn around and start to opine on monads, memory safety, and DRY principles, while their understanding of the customer is a made-up “user persona”, and they think “affordance” is the money your parents used to give you on weekends so you could go out and have a good time.

Yet others will say “Software development is theory building”. Programs are actually proofs (as in, mathematical proofs). Every commit should tell a story. And solving a customer's problem by FTPing a PHP file is a cardinal sin.

I don't mean to imply there are no developers that simultaneously care deeply about the craft of software development and really empathize with the customer. I do believe they might want to see a professional about a split personality disorder, tho.

What *is* important?

I do believe that talking to users, understanding their experience, empathizing with them, solving customers' problems and having all the stakeholders on the same page is critical to the success of a software project.

I also believe that creating good code is a craft that requires skill, patience, attention to detail, experience and wisdom, and that it will continue to be relevant in the times ahead.

¿Por qué no los dos?

To the extent that we can pull it off, I think we should aim for both. A deep understanding of the system we're building, together with a deep understanding of *why* we're building it.

Loudly proclaiming that “code is easy” or, at the opposite end, “code is art, a creative human expression that cannot be automated”, is just burying our heads in the sand.

It's cope. And you don't want cope, you want to thrive.

By this, I don't mean “jump on the LLM bandwagon.” I don't mean “become a manager of fleets of AI agents.” I also don't mean “AI-generated code is stolen slop garbage, fight it with tooth and nail, the bubble will pop soon enough anyways.”

But do recognize we're in the middle of an industry-wide tectonic change. We need to figure out how to adapt. We need to understand what is likely to change and what never changes.

What doesn't change?

Software will be getting more complex. Software will always need maintenance: bit-rot is a fact of life. So is entropy. Technology (hardware and software) will move forward, for better or worse. The tower (skyscraper?) of abstractions grows ever higher.

Users will always want more and be prepared to spend less. They still won't know how to relay their needs and wants. Worse, they still won't know exactly what they want. The disconnect between the customers (who actually pay for the software) and users (who use it) will still be here, as will the tension between the needs of the business and the needs of its customers.

Also: there will never be a shortage of snake oil salesmen. Tech du jour comes and goes (I'm still waiting for the new VR renaissance!)

What changes?

Programmers have been in the business of disrupting our own industry since the beginning. Nobody uses punch-cards any more. Very few people need to code in assembly, or COBOL. Those decades spent fighting memory bugs in C or C++, with the scars to prove it, are worthless in the age of Rust, Go, Python and JavaScript.

I'm old enough to appreciate valgrind or remember `mysql_real_escape_string()` from the PHP4 era—stuff I'll never again need in my life. And that wasn't even so long ago! I narrowly missed the dBase, Clipper, HyperCard and Access era, technologies which I can still spot operating in shops, cafes, or a dusty, once beige and now golden-brown, midi-tower still happily running some bespoke biz solution (backups? what backups?)

How do we thrive?

Accept that change happens. Be equal parts curious and critical about the new stuff.

Understand there's a lot of hype and try to discriminate between hot air and what really works (and to what extent). Also be aware of ever-shifting goalposts: stand back and look at the past year, or five, and assess the velocity of change (technical, economic, societal).

Your role and your responsibilities *will* be changing. Be willing to invest time and energy into better understanding fields or roles adjacent to yours.

If you're a senior developer, don't just find solace in deepening your expertise. Learn about user experience, customer interviews, or business strategies for the companies in your domain. It will help you gain a better appreciation of all the work done to put a piece of software into users' hands, whether or not you'll actually ever have to do any of those other bits.

If you're just starting or are junior in your role: invest in deepening your understanding of how software works. Understanding pointers, recursion, or memory hierarchy will help you even if you're a JavaScript developer. Understanding network protocols and how HTTP works will be useful even if you're building WordPress plugins. Do leetcode and learn about algorithms and data structures even if you don't need to. Don't be afraid to ask *why* and *how exactly*.

For inspiration, here are a few books and other resources that might be helpful:

- Continuous Discovery Habits

One more thing

Whoever you are, don't outsource your understanding, judgement, empathy and taste to AI. Don't abdicate your responsibility. Don't be a meat proxy.

The Night Watch

JAMES MICKENS · 11 APR 2026 · SOURCE

The Night Watch

JAMESMICKENS

James Mickens is a researcher in the Distributed Systems group at Microsoft’s Redmond lab. His current research focuses on web applications,

with an emphasis on the design of JavaScript frameworks that allow developers to diagnose and fix bugs in widely deployed web applications. James also works on fast, scalable storage systems for datacenters.

James received his PhD in computer science from the University of Michigan, and a bachelor’s degree in computer science from Georgia Tech. mickens@microsoft.com

s a highly trained academic researcher, I spend a lot of time trying to advance the frontiers of human knowledge. However, as someone who was born in the South, I secretly believe that true progress is

A

a fantasy, and that I need to prepare for the end times, and for the chickens coming home to roost, and fast zombies, and slow zombies, and the polite zombies who say “sir” and “ma’am” but then try to eat your brain to acquire your skills. When the revolution comes, I need to be prepared; thus, in the quiet moments, when I’m not producing incredible scientific breakthroughs, I think about what I’ll do when the weather forecast inevitably becomes

RIVERS OF BLOOD ALL DAY EVERY DAY. The main thing that I ponder is who will be in my gang, because the likelihood of post-apocalyptic survival is directly related to the size and quality of your rag-tag group of associates. There are some obvious people who I’ll need to recruit: a locksmith (to open doors); a demolitions expert (for when the locksmith has run out of ideas); and a person who can procure, train, and then throw snakes at my enemies

(because, in a world without hope, snake throwing is a reasonable way to resolve disputes). All of these people will play a role in my ultimate success as a dystopian warlord philosopher. However, the most important person in my gang will be a systems programmer. A person who can debug a device driver or a distributed system is a person who can be trusted in a Hobbesian nightmare of breathtaking scope; a systems programmer has seen the terrors of the world and understood the intrinsic horror of existence. The systems programmer has written drivers for buggy devices whose firmware was implemented by a drunken child or a sober goldfish. The systems program- mer has traced a network problem across eight machines, three time zones, and a brief diversion into Amish country, where the problem was transmitted in the front left hoof of a mule named Deliverance. The systems program- mer has read the kernel source, to better understand the deep ways of the universe, and the systems programmer has seen the comment in the sched- uler that says “DOES THIS WORK LOL,” and the systems programmer has wept instead of LOLed, and the systems programmer has submitted a kernel patch to restore balance to The Force and fix the priority inversion that was causing MySQL to hang. A systems programmer will know what to do when society breaks down, because the systems programmer already lives in a world without law.

Listen: I’m not saying that other kinds of computer people

are useless. I believe (but cannot prove) that PHP developers have souls. I think it’s great that database people keep trying to improve select-from-where, even though the only queries that cannot be expressed using select-from-where are inap- propriate limericks from “The Canterbury Tales.” In some way that I don’t yet understand, I’m glad that theorists are investigating the equivalence between five-dimensional Tur- ing machines and Edward

Scissorhands. In most situations, GUI designers should not be forced to fight each other with tridents and nets as I yell “THERE ARE NO MODAL DIA- LOGS IN SPARTA.” I am like the Statue of Liberty: I accept everyone, even the wretched and the huddled and people who enjoy Haskell. But when things get tough, I need mission-crit- ical people; I need a person who can wear night-vision goggles and descend from a helicopter on ropes and do classified things to protect my freedom while country music plays in the background. A systems person can do that. I can realistically give a kernel hacker a nickname like “Diamondback” or “Zeus Hammer.” In contrast, no one has ever said, “These semi-

transparent icons are really semi-transparent! IS THIS THE WORK OF ZEUS HAMMER?”

I picked that last example at random. You must believe me when I say that I have the utmost respect for HCI people.

However, when HCI people debug their code, it’s like an

art show or a meeting of the United Nations. There are tea breaks and witticisms exchanged in French; wearing a non- functional scarf is optional, but encouraged. When HCI code doesn’t work, the problem can be resolved using grand theo- ries that relate form and perception to your deeply personal feelings about ovals. There will be rich debates about the socioeconomic implications of Helvetica Light, and at some point, you will have to decide whether serifs are daring state- ments of modernity, or tools of hegemonic oppression that implicitly support feudalism and illiteracy. Is pinching-and- dragging less elegant than circling-and-lightly-caressing?

These urgent mysteries will not solve themselves. And yet, after a long day of debugging HCI code, there is always hope, and there is no true anger; even if you fear that your drop- down list should be a radio button, the drop-down list will suffice until tomorrow, when the sun will rise, glorious and

vibrant, and inspire you to combine scroll bars and left-click- ing in poignant ways that you will commemorate in a sonnet when you return from your local farmer’s market.

This is not the world of the systems hacker. When you debug a distributed system or an OS kernel, you do it Texas-style. You gather some mean, stoic people, people who have seen things die, and you get some primitive tools, like a compass and a rucksack and a stick that’s pointed on one end, and you walk into the wilderness and you look for trouble, possibly while

using chewing tobacco. As a systems hacker, you must be pre- pared to do savage things, unspeakable things, to kill runaway threads with your bare hands, to write directly to network ports using telnet and an old copy of an RFC that you found in the Vatican. When you debug

systems code, there are no high- level debates about font choices and the best kind of turquoise, because this is the Old Testament, an angry and monochro- matic world, and it doesn't matter whether your Arial is Bold or Condensed when people are covered in boils and pestilence and Egyptian pharaoh oppression. HCI people discover bugs by receiving a concerned email from their therapist. Systems people discover bugs by waking up and discovering that their first-born children are missing and "ETIMEDOUT " has been written in blood on the wall.

What is despair? I have known it—hear my song. Despair is when you're debugging a kernel driver and you look at a mem- ory dump and you see that a pointer has a value of 7. THERE IS NO HARDWARE ARCHITECTURE THAT IS ALIGNED ON

7. Furthermore, 7 IS TOO SMALL AND ONLY EVIL CODE WOULD TRY TO ACCESS SMALL NUMBER MEMORY.

Misaligned, small-number memory accesses have stolen decades from my life. The only things worse than misaligned, small-number memory accesses are accesses with aligned buf- fer pointers, but impossibly large buffer lengths. Nothing ruins a Friday at 5 P.M. faster than taking one last pass through the log file and discovering a word-aligned buffer address, but a buffer length of NUMBER OF ELECTRONS IN THE UNI-

VERSE. This is a sorrow that lingers, because a 2893 byte read is the only thing that both Republicans and Democrats agree is wrong. It's like, maybe Medicare is a good idea, maybe not, but there's no way to justify reading everything that ever existed a jillion times into a mega-jillion sized array. This constant war on happiness is what non-systems people do not understand about the systems world. I mean, when a machine learning

algorithm mistakenly identifies a cat as an elephant, this is

actually hilarious. You can print a picture of a cat wearing an elephant costume and add an ironic caption that will entertain people who have middling intellects, and you can hand out cop- ies of the photo at work and rejoice in the fact that everything is still fundamentally okay. There is nothing funny to print when you have a misaligned memory access, because your machine is dead and there are no printers in the spirit world.

An impossibly large buffer error is even worse, because these errors often linger in the background, quietly overwriting your state with evil; if a misaligned memory access is like a criminal burning down your house in a fail-stop manner, an impossibly large buffer error is like a criminal who breaks into your house, sprinkles sand atop random bedsheets and toothbrushes, and then waits for you to slowly discover that your world has been tainted by madness. Indeed, the common discovery mode for an impossibly large buffer error is that your program seems to

be working fine, and then it tries to display a string that should say “Hello world,” but instead it prints “#a[5]:3!” or another syntactically correct Perl script, and you’re like WHAT THE HOW THE, and then you realize that your prodigal memory accesses have been stomping around the heap like the Incred- ible Hulk when asked to write an essay entitled “Smashing Considered Harmful.”

You might ask, “Why would someone write code in a grotesque language that exposes raw memory addresses? Why not use

a modern language with garbage collection and functional programming and free massages after lunch?” Here’s the answer: Pointers are real. They’re what the hardware under- stands. Somebody has to deal with them. You can’t just place a LISP book on top of an x86 chip and hope that the hardware learns about lambda calculus by osmosis. Denying the exis- tence of pointers is like living in ancient Greece and denying the existence of Krackens and then being confused about why none of your ships ever make it to Morocco, or Ur-Morocco,

or whatever Morocco was called back then. Pointers are like Krackens—real, living things that must be dealt with so that polite society can exist. Make no mistake, I don’t want to write systems software in a language like C++. Similar to the Necro- nomicon, a C++ source code file is a wicked, obscure document that’s filled with cryptic incantations and forbidden knowl- edge. When it’s 3 A.M., and you’ve been debugging for 12 hours, and you encounter a virtual static friend protected volatile

templated function pointer, you want to go into hibernation and awake as a werewolf and then find the people who wrote the C++ standard and bring ruin to the things that they love. The C++ STL, with its dyslexia-inducing syntax blizzard of colons and angle brackets, guarantees that if you try to declare any reasonable data structure, your first seven attempts will result in compiler errors of Wagnerian fierceness:

Syntax error: unmatched thing in thing from std::nonstd::_map<_Cyrillic, _$$$dollars>const basic_string< epic_ mystery,mongoose_traits < char>, _default_alloc_<casual_ Fridays = maybe>>

One time I tried to create a list<map<int>>, and my syntax errors caused the dead to walk among the living. Such things are clearly unfortunate. Thus, I fully support high-level lan- guages in which pointers are hidden and types are strong and the declaration of data structures does not require you to solve a syntactical puzzle generated by a malevolent extraterrestrial species. That being said, if you find yourself drinking a martini and writing programs in

garbage-collected, object-oriented Esperanto, be aware that the only reason that the Esperanto runtime works is because there are systems people who have exchanged any hope of losing their virginity for the exciting opportunity to think about hex numbers and their relationships

with the operating system, the hardware, and ancient blood rituals that Bjarne Stroustrup performed at Stonehenge.

Perhaps the worst thing about being a systems person is that other, non-systems people think that they understand the daily tragedies that compose your life. For example, a few weeks ago, I was debugging a new network file system that my research group created. The bug was inside a kernel-mode component, so my machines were crashing in spectacular and vindic-

tive ways. After a few days of manually rebooting servers, I had transformed into a shambling, broken man, kind of like a computer scientist version of Saddam Hussein when he was pulled from his bunker, all scraggly beard and dead eyes and florid, nonsensical ramblings about semi-imagined enemies.

As I paced the hallways, muttering Nixonian rants about my code, one of my colleagues from the HCI group asked me what my problem was. I described the bug, which involved concurrent threads and corrupted state and asynchronous message delivery across multiple machines, and my coworker said,

“Yeah, that sounds bad. Have you checked the log files for errors?” I said, “Indeed, I would do that if I hadn’t broken every component that a logging system needs to log data. I have a network file system, and I have broken the network, and I have broken the file system, and my machines crash when I make eye contact with them. I HAVE NO TOOLS BECAUSE I’VE DESTROYED MY TOOLS WITH MY TOOLS. My only logging

option is to hire monks to transcribe the subjective experience of watching my machines die as I weep tears of blood.” My co- worker, in an earnest attempt to sympathize, recounted one of his personal debugging stories, a story that essentially involved an addition operation that had been mistakenly replaced with

a multiplication operation. I listened to this story, and I said, “Look, I get it. Multiplication is not addition. This has been known for years. However, multiplication and addition are at least related. Multiplication is like addition, but with more addition. Multiplication is a grown-up pterodactyl, and addi- tion is a baby pterodactyl. Thus, in your debugging story, your code is wayward, but it basically has the right idea. In contrast, there is no family-friendly GRE analogy that relates what my code should do, and what it is actually doing. I had the mod-

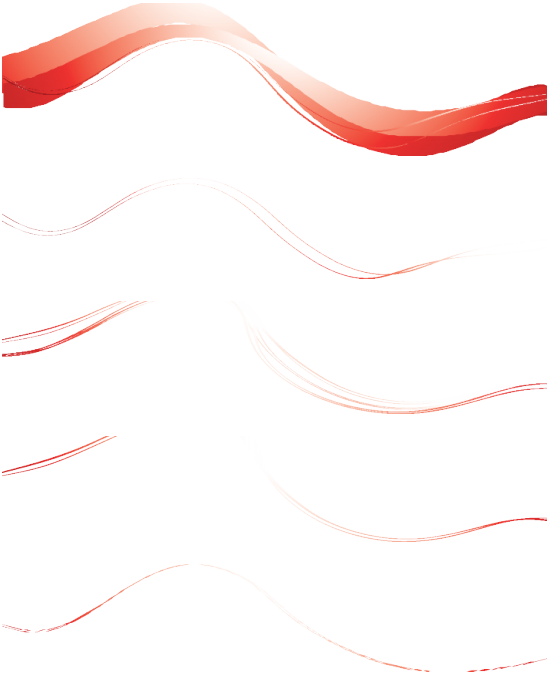
est goal of translating a file read into a network operation, and now my machines have tuberculosis and orifice containment issues. Do you see the difference between our lives? When you asked a girl to the prom, you discovered that her father was a cop. When I asked a girl to the prom, I DISCOVERED THAT HER FATHER WAS STALIN.”

In conclusion, I’m not saying that everyone should be a systems hacker. GUIs are useful. Spell-checkers are useful. I’m glad that people are working on new kinds of bouncing icons because they believe that humanity has solved cancer and homelessness and now lives in a consequence-free world

of immersive sprites. That’s exciting, and I wish that I could join those people in the 27th century. But I live here, and I live now, and in my neighborhood, people are dying in the streets. It’s like, French is a great idea, but nobody is going to invent French if they’re constantly being attacked by bears. Do you see? SYSTEMS HACKERS SOLVE THE BEAR MENACE.

Only through the constant vigilance of my people do you get

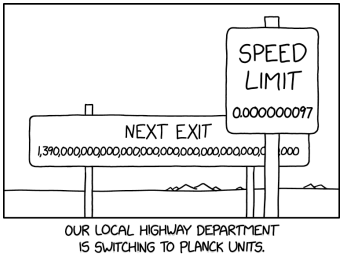
the freedom to think about croissants and subtle puns involv- ing the true father of Louis XIV. So, if you see me wandering the halls, trying to explain synchronization bugs to confused monks, rest assured that every day, in every way, it gets a little better. For you, not me. I’ll always be furious at the number 7, but such is the hero’s journey.



usenx

Highway Signs

XKCD.COM · 25 AUG 2026 · SOURCE



Highway engineers like Planck units because, like the speed of light, the energy capacity of a typical car's gas tank is 1.

Conceptual integrity and counting lines of code

SIMON WILLISON'S WEBLOG: ENTRIES · 19 AUG 2026 · SOURCE

Conceptual integrity and counting lines of code

Last week I recorded an episode of the Talking Postgres podcast with Claire Giordano on the subject of “How AI is changing software development”. We had a really great conversation. Here are a couple of my highlights from a lightly edited transcript (prompt to Claude: “very minor edits to remove disfluencies”).

This is the latest version of an argument I’ve been trying to build about why sometimes it *does* make sense to talk about lines of code as an indicator of productivity with coding agents, at 35:01:

> A lot of people will tell you it makes no sense to measure productivity in lines of code. I’d actually disagree, because there’s a hard limit. In the before-times, a software engineer could produce a few hundred lines of production-ready code per day — and 200 lines of working, debugged, production-level code is an incredibly good day. Most days you’d produce 50 or 60.
>
> If agents let you produce a thousand lines of debugged code, that really is a very meaningful improvement — as long as the code is the same quality: maintainable, tested, all of that. You can get to that point with agents, but it takes a huge amount of skill and knowledge and experience. That’s what senior engineers are made of.

> I can do way more work as a single engineer than I could without agents. So you could argue, why should a company have more than one engineer? Beyond the obvious bus factor thing — a team of one is a very badly designed team — the answer is that the new limiting factor is cognitive capacity. I can churn out code a hundred times faster. I don't have the cognitive capacity to stay on top of 100 times the amount of code. So you still need a team of engineers, so you can load balance that cognitive capacity across the team.

And this section on conceptual integrity at 46:03, which Claire equated to the Winchester Mystery House!

> **Simon**: There's a concept in *The Mythical Man-Month* — conceptual integrity — where well-designed software has an integrity to it: there are no surprises in it, it covers exactly the right domain of things, everything fits together and makes sense. That's so much harder with coding agents, where you can have an idea for a feature, run a prompt, and five minuteslater you've got the feature. Your software grows little weird bumps in funny different directions.
>
> **Claire**: You know my analogy for that? The Winchester Mystery House.
>
> **Simon**: It's got 140 rooms, because the woman who built it was the widow of the guy who invented the Winchester rifle, and her psychic told her she'd be haunted by the ghosts of everyone killed with that rifle unless she kept building the house forever. So for 40 years she kept adding new rooms. That's exactly the problem with coding agents and software: it's very easy to keep adding new rooms, because the cost of adding those rooms is so much cheaper. What you end up with is something where the conceptual integrity falls apart — and then it's harder to make decisions about it.
>
> It all keeps coming back to discipline. It used to be that the discipline was enforced on you by the amount of time it took. You'd come up with an idea for a crazy feature and think "yeah, but that would take me a week — I cannot justify that, so I'll forget about it." If it takes an hour, it's so much easier to justify.

(Side-note: the Wikipedia article includes credible sources that dispute the story about the psychic.)